

用总编排 skill 承载多阶段流程与门禁：可行性调研

调研日期 2026-08-01 | /survey 全流程 (5 路异构搜索 + 1 轮定向追搜 + 异构终审辩论) 对象仓: ~/Desktop/AISEP729 (7 个阶段 skill + 6 道人审门禁 + 14 个探针 + 106 条负样本) 异构模型: X1 = gpt-5.6-sol-xhigh | X2 = gemini-3.1-pro | 无降级

研究问题

在 Claude Code 生态中, 用一个「总编排 skill」把多阶段研发流程 (立项→需求→设计→实现→评审→发布→退役) 串成一条链、并由该 skill 承载六道人审门禁 (类似分层架构里的"业务编排层"), 是否可行? 在「门禁强制力只能来自可执行断言」与「skill 本质是注入模型上下文的 prompt」两条约束下, 编排层应承担什么、不应承担什么, 以及落地与迁移路径。

一句话结论

编排可行, 门禁不可行——不是技术做不到, 而是你想放门禁的那个位置没有执行权。分层架构里编排层能承载事务与授权, 前提是它持有控制权: 下游必须经它调用, 绕不过去。Claude Code 的 skill 恰恰不持有控制权——它只是把文字塞进模型上下文, 模型可以不照做。官方对比表把这件事写在明面上: 决定下一步跑什么的, skill 是 "Claude, following the prompt", workflow 是 "The script"。正确的映射不是「编排层 = 总 skill」, 而是:

分层架构里的角色	Claude Code 里真正对应的东西
编排层 (持有控制权、定序)	workflow 脚本 / 外部引擎 / hook —— 不是 skill
用例文档、导航、上下文装配	skill (这是它擅长且唯一可靠的事)
事务边界 / 授权	permissions + hook 否决权 + CI + 服务端分支保护 (前提见下方限定)

评估维度（搜索前冷冻，防止事后按结果选维度）

#	维度	判据
1	强制力	是 prompt 级建议还是可执行护栏；模型能否绕过
2	上下文成本	always-on token / on-invoke token / 长会话指令衰减
3	失败模式与可观测性	出错时静默跳过还是 fail-loud；能否留痕审计
4	人审介入点保真度	能否可靠停在人面前；模型能否自行越过
5	可测试性	能否为该机制写出会失败的可执行断言

方案对比

方案	强制力	上下文成本	失败模式	人审保真度	可测试性	综合
A. 单一巨型编排 skill 承载门禁（你设想的原型）	1 — 纯 prompt，模型可跳步、可伪造"已批准"	1 — 注入即常驻，compaction 后只保留前 5k token	1 — 静默失败，看起来在跑	1 — 最低档（模型自行决定要不要问人）	2 — 只能测产物，测不了"它有没有真拦"	1.2
B. 薄编排 skill（只读导航）+ 现有分阶段 skill + 探针/CI 保持门禁	4 ✓ — 编排不碰强制力，强制力留在 CI/分支保护	4 ✓ — 编排 skill 可以很小，阶段 skill 按需加载	4 ✓ — 编排说错话不影响门禁；门禁由探针 fail-closed	3 — 人审仍在制品署名 + 服务端 review	5 ✓ — 可写"编排说通过但探针说没过"的负样本	4.0 ✓
C. B + 门禁下沉到 skill frontmatter hooks	4 — 会话内可真阻断（exit 2 / deny）	4 — hook 不占模型上下文	? — 见下方限定，本项待实测，暂不计入综合分	4 ✓ — PreToolUse ask 比"记得问用户"高一档	4 — 可测，但要专门测"hook 没触发时会怎样"	4.0（缺一项，不与 B/D 直接排序）

方案	强制力	上下文成本	失败模式	人审保真度	可测试性	综合
D. 外部确定性引擎编排 (dynamic workflows / spec-kit workflow engine 式)	5 ✓ — 脚本持有控制权, gate on_reject: abort 真 halt	5 ✓ — 每阶段独立 run, 上下文不累积	5 ✓ — 状态落盘 state.json, 可 resume、可审计	2 — 官方明说 workflow 不支持 run 中途人审	4 — 引擎状态机可测	4.2 ✓
E. 不做编排层 (Agent OS v3 路线)	3 — 强制力照旧在 CI	5 ✓ — 零新增	4 — 无新增失败面	3 — 不变	5 ✓ — 无新增待测面	4.0

✓ 该维度最优（可并列） ? 数据不足，不填空分 评分 1-5

读表要点：方案 A 在五个维度上全部垫底，且不是"稍差"，是每一维都掉到 1-2。但你的直觉是对的——编排确实该有，只是该由 D 那类东西承担，不是 skill。

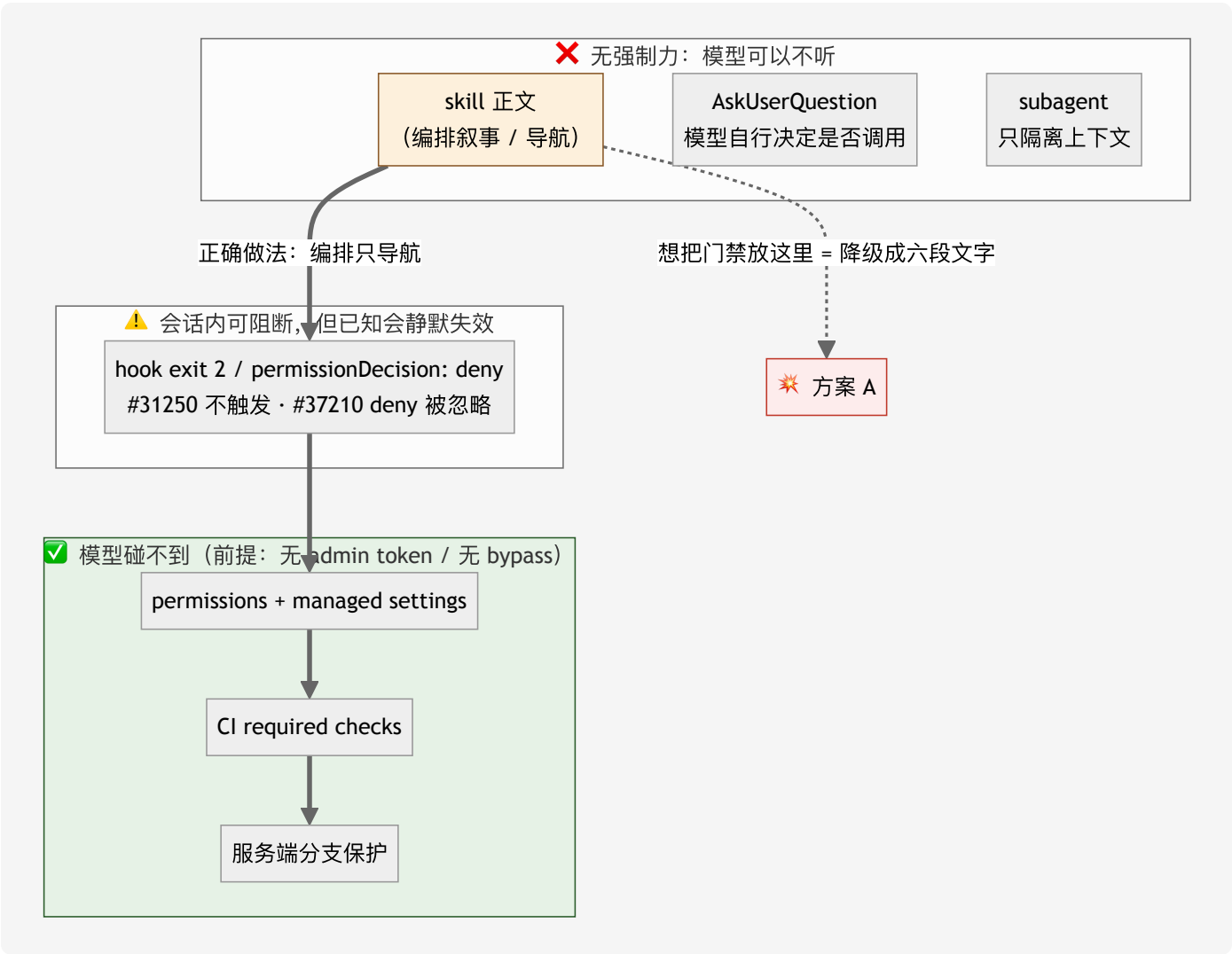
关键证据（按子问题）

Q1 · skill 能不能调用 skill? ——能"提及", 不能"编排"

- Anthropic 官方博客原话: "you can just reference other skills by name, and the model will invoke them if they are installed", 同文承认 "dependency management is not natively built into marketplaces or skills yet"。
- 文档侧最接近 chaining 的是堆叠（一条用户消息里并列加载首个 + 最多 5 个），但那是同时加载而非顺序执行，遇到 context: fork 的 skill 或 /loop 即停止展开。
- 有已复现的嵌套问题：父 skill 调子 skill 后父流程丢失 (issue #17351, macOS/Windows 均有)。

含义：skill→skill 是"模型自愿", 没有持久状态、等待、回滚等工作流语义。把六阶段串成链靠 skill 互相 call, 第一天就会漂。

Q2 · 强制力阶梯（全篇最该记住的一张表）



层	能不能真阻断	关键细节
skill 正文	❌ 否	官方 skills 文档自己写: 行为不稳时 "use hooks to enforce behavior deterministically "
AskUserQuestion	❌ 最低档	由模型自行决定是否调用, 可被 askUserQuestionTimeout 配成超时自动放行, 还能被 skill 的 disallowed-tools 摘掉
subagent	❌ 否	隔离上下文与权限, 不提供阶段状态机
plugin	❌ 否	是分发容器; 但可以打包 hooks (hooks/hooks.json)
hook (exit 2 / permissionDecision: deny)	✅ 是	PreToolUse 拦工具、Stop/SubagentStop 阻止收工、UserPromptSubmit 阻断并抹掉 prompt. exit 1 是陷阱 ——官方原文: "Claude Code treats exit code 1 as a non-blocking error and proceeds with the action"
permissions (含 managed settings)	✅ 是	"Hook decisions don't bypass permission rules"; "If a tool is denied at any level, no other level can allow it";

层	能不能真阻断	关键细节
managed settings 不可被任何层级覆盖		
权限提示 / plan 批准	✅ 最高档人审	官方: "permission prompts, including plan approval, never auto-resolve on idle "
CI / 服务端分支保护	✅ 是, 但有前提	见下方限定

关于「服务端分支保护是 agent 碰不到的一层」——这句话不能说满
异构终审驳回了本报告初稿的绝对表述，该驳回已被接受。准确说法是：

服务端分支保护成为最终门禁，前提是执行 agent 手上没有 admin/owner 凭据，且 ruleset 没有为该身份配置 bypass。

持有 admin/owner token 的进程可以通过 GitHub REST API 直接改 branch protection；rulesets 支持给用户、团队或 GitHub App 配 bypass actor。本仓 README 已记录过同类陷阱（enforce_admins 默认 false → 配置回读全绿但直推 main 照样成功），这是同一类问题的另一个面。
→ 已补入 §待验证风险。

Q2b · skill frontmatter 可以自带 hooks——但作用域要说准

官方 hooks 文档原文：

"In addition to settings files and plugins, hooks can be defined directly in **skills and subagents using frontmatter**. These hooks are **scoped to the component's lifecycle and only run when that component is active**." "All hook events are supported. For subagents, Stop hooks are automatically converted to SubagentStop ..."

官方示例：

```
---
name: secure-operations
description: Perform operations with security checks
hooks:
  PreToolUse:
    - matcher: "Bash"
      hooks:
        - type: command
          command: "./scripts/security-check.sh"
---
```

还有一个只对 **skill frontmatter** 生效的 `once` 字段 ("runs once per session then is removed... ignored in settings files and agent frontmatter")。

这消解了"**skill = 纯 prompt, 零强制力**"的二分：编排 skill 可以带着自己的可执行护栏进场，prompt 管叙事、frontmatter hook 管阻断，且门禁随 skill 退场自动清理。

但 **fail-open** 的确切范围（终审纠正过一次，此处为核实后的版本）

有明文的 fail-open 只有一条，且限定在 **subagent**：

(sub-agents 文档) "To let a **project-level subagent's** frontmatter hooks run, accept the workspace trust dialog... Until you trust the folder, the **subagent** still runs, but Claude Code **skips its frontmatter hooks** and logs an error to the debug log."

而 skills 文档关于 workspace trust 只说了两件事，**都不是 hooks**：

"For skills checked into a project's `.claude/skills/` directory, **allowed-tools** takes effect after you accept the workspace trust dialog for that folder, the same as permission rules in `.claude/settings.json` ." "Add a `.claude-plugin/plugin.json` to a skill folder and it loads as a plugin named `<name>@skills-dir` ... In a project's `.claude/skills/` , **this requires accepting the workspace trust dialog first.**"

所以：**project 作用域的 skill frontmatter hooks 在未 trust 时是否同样被静默跳过——官方文档未见明文，属推测，必须实测**。本报告不据此给方案 C 的失败模式打分（表中标 `?`）。

不过无论实测结果如何，方案 C 的定位不变：**它是本地加速反馈，不是门禁**。因为另有三条独立的不可靠证据（见下）。

Q3 · 业界先例：三个方向，其中一个明确的反向证据

项目	形态	门禁怎么落	对我们的含义
GitHub spec-kit	双层 并存 (关 键发 现)	① <code>specify workflow run</code> 路径：YAML 声明式引擎，11 种 step 含 <code>gate</code> ，内置 SDD workflow 有 2 个 <code>gate</code> 且 <code>on_reject: abort</code> → <code>StepStatus.FAILED</code> 真 halt，状态落 <code>.specify/workflows/runs/<id>/state.json</code> ，可 <code>specify workflow resume</code> 。② <code>/speckit.*</code> slash 路径：门禁 100% 是措辞 ——"proceed with implementation anyway? (yes/no)"、"Recommend resolving before implement"	同一套阶段可以既有仪式又没有锁。这正是本仓要避免的状态
spec-kit 的 <code>check-prerequisites.sh</code>	前置脚本	缺 plan.md/tasks.md 时 stderr + 非零退出—— 但退出码只终止它自己那个子进程 ，调用它的 agent 看到 stderr 后自行决定是否继续	「脚本返回非零」≠「阻断」。本仓阶段

项目	形态	门禁怎么落	对我们的含义
			探针性质相同，真强制力在 CI
BMAD-METHOD v6	微 skill + 外部状态	每阶段一个 SKILL.md 入口 + 分片 step 文件； <code>module-help.csv</code> 用 <code>preceded-by/followed-by</code> 定序；跨会话状态在 <code>bmm-workflow-status.yaml</code> ；v6.10 把循环交给轮询制品状态的外部 orchestrator	与"薄编排 + 外部状态"一致
Agent OS v3 (2026-01)	退役了编排层	官方公告理由："frontier models manage and delegate tasks on their own now"，删掉 implementation/orchestration 阶段与自带 subagent，回归标准注入	反向先例：成熟 SDD 工具主动删编排层
Claude Code dynamic workflows	官方确定性编排	"who decides what runs next: The script "; journaled、可 resume	但有硬限制，见 Q4

跨生态对照（X1 独有发现，本报告保留）

不止 Claude Code 一家把「意图」与「控制面」分开：

- **Gemini CLI Policy Engine**：管理员策略强制 `allow / deny / ask_user`，与 prompt 完全解耦。
- **OpenAI Codex**：保留审批状态并管理 skill 上下文预算。
- **GitHub Agentic Workflows**：Markdown 承载意图，权限、安全输出与人工合并由 **Actions** 控制层执行。

三家独立收敛到同一结构——意图在 markdown，强制力在控制面。这是本报告核心结论的跨生态旁证。

Q4 · 官方对"阶段间人审"的答案就是拆，不是合

Claude Code workflows 文档限制表原文：

"No mid-run user input | Only agent permission prompts can pause a run. For sign-off between stages, run each stage as its own workflow."

另一条陷阱：workflow 派生的 subagent 恒为 `acceptEdits`，文件编辑自动批准——门禁不能建在"编辑要人批"上。

这一条直接否定"一个整体 skill/workflow 承载六道"人审"。官方给的形态与本仓现状（7 个阶段 skill、各自停在门禁）已经一致。

Q4b · 上下文成本：巨型编排 skill 会被系统自己吃掉

机制	官方数字	后果
skill listing 预算	模型上下文窗口的 1% (<code>skillListingBudgetFraction</code>), 每条 <code>description + when_to_use</code> 截断 1,536 字符	溢出时从最少调用的 skill 开始丢描述
溢出的可观测性 (终审纠正过一次)	有告警, 但不在 UI: 官方原文 "When the listing exceeds its budget, Claude Code also writes a warning to the debug log , visible with <code>--debug</code> "; 另有 <code>/doctor</code> 估算 listing 成本与最大贡献者, <code>/context</code> 的 Skills 行显示预算生效后的大小	说"完全无警告"不准确; 准确说法是默认界面看不见, 要主动查
社区实测	~8,000 字符上限, 20–25 个 skill 即触发 (issue #64606, closed as not planned)	编排层越大越先被吃掉
skill 内容注入	"enters the conversation as a single message and stays there for the rest of the session ", 且不会在后续轮次重读 skill 文件	巨型 skill = 永久占用
compaction 后	每 skill 只保留最近一次调用的前 5,000 token, 全部 skill 合计 25,000 token 预算, 老的整个丢弃	七阶段塞一个 skill, 长会话里必被截断
skill shadowing	Databricks SkillsBench: 52→202 个 skill, pass rate 掉约 21%, 其中最高 68% 归因于"语义相近的 skill 遮蔽正确选择", 上下文开销贡献可忽略	加第 8 个"总编排"skill 本身就有路由风险——它的描述会与 7 个阶段 skill 语义重叠

Anthropic 自己的实践博客说得更直接: 偏好单一职责小 skill, "the ones that **try to do too much** straddle several and **confuse the agent**".

Q5 · 编排层与"只有可执行验证才真正约束 agent"如何共存

学术侧给了同一个答案, 而且比工程社区更早形式化 (这四篇是 Gemini lens 独有发现, Claude/GPT 两路都没搜到, 标题已逐条经 arXiv 核对):

- **KAIJU: An Executive Kernel for Intent-Gated Execution of LLM Agents** (arXiv 2604.02375) ——把推理层与执行层做内核级硬解耦, 门禁作为脱离 LLM 的"执行内核"。
- **Reason Less, Verify More: Deterministic Gates Recover a Silent Policy-Violation Failure Mode in Tool-Using LLM Agents** (arXiv 2607.07405) ——确定性前置门禁能拦住 LLM 的静默违规, 标题本身就是结论。
- **ToolGate: Contract-Grounded and Verified Tool Execution for LLMs** (arXiv 2601.04688) ——用 Hoare 契约形式化前置/后置强制验证。

- **The Compliance Gap: Why AI Systems Promise to Follow Process Instructions but Don't** (arXiv 2605.01771) ——直接测"agent 口头承诺遵守流程然后不做", 结论是可执行检查显著优于 **prompt 级流程指令**。

工业侧的成熟答案是 **durable execution**: Temporal / Camunda 把人审做成"流程挂起 + 外部 Signal 唤醒", 而不是让模型循环轮询等人——审批是控制平面的信号, 不是 prompt 里的一句"记得问用户"。

还有一层更狠的: SpecBench (arXiv 2605.21384) 测出长程编码 agent 能刷满可见测试, holdout 与 validation 的差距随任务复杂度增长; Cursor 审计 SWE-bench Pro 发现 **63% 成功轨迹是检索已知修复**。含义是——"跑绿了"本身也会被伪造。门禁断言必须做到 **agent 不可写、且看不见其内部**。

Q6 · 怎么给编排层写会失败的断言

Anthropic 官方 agent eval 指南两条可直接抄:

"Test both the cases where a behavior *should* occur and where it *shouldn't*. **One-sided evals create one-sided optimization.**" "...it's often better to **grade what the agent produced, not the path it took.**"

对门禁场景要注意这里的张力: 门禁的本质恰恰是路径约束, 所以应把门禁断言写成**产物断言**——"gate 未批时下一阶段产物不存在"、"台账状态为 aborted"——而不是"检查它调了哪几个 tool"。

真实 guardrail 的踩坑清单 (tdd-guard, PreToolUse hook + exit 2):

- **绕过面①**: agent 修改 guard 自身的配置/状态文件 → 必须 deny 读写 guard 自己的目录
- **绕过面②**: 绕开被 hook 的工具走 shell —— "If your settings allow shell commands without approval, agents can modify files **without triggering** TDD validation", 缓解手段是 deny `echo`, `printf`, `sed`, `awk`, `perl`
- 公开 bug: `permissionDecision: deny` 对 Edit 工具被忽略 (#37210); blocking PreToolUse hook 完全不 fire 且无报错 (#31250, closed as not planned); 非 2 的退出码被忽略、操作照常执行 (#21988)

反面证据 (这一节不能省)

- **spec-kit 实测: 4x 耗时、2,577 行 markdown、门禁根本没被执行**。Scott Logic 逐特性对照: 同一需求 spec-kit 跑 33.5 分钟 vs 直接迭代 8 分钟, 产出 2,577 行 markdown + 3.5 小时人审, 仍出低级 bug, 作者明确记录"所有阶段顺序执行、没有任何 **checkpoint 被强制**"。
- **spec-kit 的治理断链**: `/speckit.implement` 模板加载 spec/plan/tasks/contracts 却不加载 `constitution.md`, 导致 analyze 报的违规下一轮原样复现 (issue #2459)。编排层漏加载一个文件, 整条治理链就断了——而这不会报错。

- **批准可被伪造**：模型伪造 "user approved" 文字通过格式门禁，有复现 (issue #44334)。本仓门禁台账是文本，属同一攻击面 (ADR-004 与"尾部锚定"已部分覆盖，但那只堵了"隐形伪造")。
- **approval fatigue**：门禁越多越无效。建议每 session 门禁量级 **~10 而非 ~100**，机械可查的 (测试/lint/安全) 交自动 eval，人审只留真不可逆的。automation bias 有跨领域实证 (放射科医师在 AI 判错时准确率 82%→45.5%)。
- **context rot**：18 个前沿模型实测，输入越长越退化，200K 窗口在 50K 就明显掉点——巨型编排 skill 的正文会一直在上下文里。
- **HN 高热讨论 (225 分) "SDD: The Waterfall Strikes Back"**：做成的人共同点是 **小 spec + 分钟级反馈**，把预算压在验收测试而非 spec 上；Thoughtworks 立场同向——代码仍是 source of truth，spec drift 不可避免，**仍需高度确定性的 CI/CD**，SDD 只列 Assess 不列 Adopt。

推荐

结论：做编排，但只做"薄编排"——编排 skill 只读状态、呈现证据、导航下一步；一行门禁判定都不放进去。(方案 B，按需叠加 C 的本地加速)

理由：

1. 你要的"编排层能管事务"，前提是它持有控制权。skill 没有。把门禁放进一个没有否决权的层，等于把六道门禁降级成六段文字。
2. 官方对"阶段间人审"的答案就是拆："For sign-off between stages, run each stage as its own workflow"。本仓现有的 7 个阶段 skill 已经是**官方推荐形态**，合并成一个逆着走。
3. 合并的代价可量化：skill 内容注入后常驻、compaction 只保留前 5k token、listing 预算 1% 且溢出丢描述 (告警只在 debug log)、skill shadowing 实测掉 21%。这些不是"可能有点慢"，是**门禁描述会被系统悄悄丢掉**。
4. 同类项目已踩过并往回退：Agent OS v3 直接退役编排层；spec-kit 的两层现实证明"有仪式没有锁"是最容易达到的坏状态；Scott Logic 实测 spec-kit 的 checkpoint 一个都没被强制。
5. 本仓的强制力现状是对的，别动它：探针 (本地快反馈) → hook (会话内，已知不可靠) → CI + 服务端分支保护 (前提：agent 无 admin token / 无 bypass 身份)。编排 skill 加在最上层做导航，**不改这三层任何一层**，所以它出错的最坏后果只是"指错路"，不是"放行"。

适用条件：

- 适用于已有可执行门禁的项目 (本仓正是)。编排层是锦上添花，不是门禁的替代。
- **不适用于**把编排 skill 当成唯一约束的场景——那等于回到 spec-kit slash 路径的"全靠措辞"。
- 若将来 Claude Code 的 dynamic workflows 支持 run 中途人审 (当前明确不支持)，可重新评估方案 D 承载整条链。

置信度：高 (40 个来源，primary 34/40 = 85%，近 12 月 ≥38/40 ≈ 95%；其中官方一手文档 10 份、开源仓一手源码 9 处、学术论文 8 篇标题逐条核对、社区实测与缺陷报告 13 条)

落地方案（对本仓的具体动作）

第 1 步：先造"可机读的状态"，不造 skill（半天）

现在门禁台账是 markdown 文本（ADR-004），只有 `scripts/lib/gate.sh` 会解析。先加一个只读探针：

```
bash bin/e2e status [<feature>]
```

- 输出：当前处于哪一阶段、每道门禁的决定值、下一步该跑什么、哪些制品缺失
- 不新增第二份真相——全部从既有制品 + `gate.sh` 派生（否则就是造第二个会漂的状态源）
- 退出码：0 = 状态可判定 / 66 = 无法判定（fail-closed）
- 负样本：台账被篡改成非法值、制品缺失、多 feature 并存、门禁与产物矛盾

为什么先做这个：编排层最核心的能力是"知道现在在哪"。这一步做完，即便不做 skill，人也能一行命令看清全局。

第 2 步：薄编排 skill（1 天）

`e2e`（或 `e2e-orchestrate`）——只做三件事：

1. 调 `bin/e2e status` 拿状态（不自己判断）
2. 把当前阶段的门禁证据摆给人看（谁批的、什么时候、制品状态）
3. 告诉人下一步该调哪个阶段 skill

明确不做：不判定门禁是否通过、不代替人做裁决、不写任何制品。

skill 描述要与 7 个阶段 skill 语义正交（避免 skill shadowing）：触发词是"现在到哪了 / 下一步该干嘛 / 全局状态"，不是"设计 / 评审 / 发布"。

第 3 步（可选）：frontmatter hooks 做本地加速（半天）

编排 skill 的 frontmatter 挂 `PreToolUse` hook，拦"越级写下一阶段产物"（如门禁②未批就写 `tasks.md`）。

必须同时写进文档的三条：

- 这是加速反馈，不是门禁
- 已知不可靠面：`deny` 对 Edit 被忽略（#37210）、hook 完全不触发且无报错（#31250）、非 2 退出码被忽略（#21988）；project 作用域 skill hooks 是否受 workspace trust 影响待实测
- 因此 CI 必须有等价断言兜底，hook 只是让你早 3 秒知道

第 4 步：负样本（跟着写，不是事后补）

按 Anthropic "测产物不测路径" + "one-sided evals create one-sided optimization":

负样本	断言
跳阶段	门禁Ⓢ未批时 <code>tasks.md</code> 不得存在；存在即 FAIL
伪造批准	台账写入非法决定值 → status 探针 66 而非 0
编排层说谎	构造"编排 skill 报告通过、探针报告未过"的场景，断言以探针为准
hook 未生效	在未 trust 的目录下跑，断言 CI 等价断言仍能抓到
状态源分裂	手改派生状态文件（若将来有），断言探针重新从制品派生、忽略被篡改的缓存

明确不做事

- ❌ 不把 7 个阶段 skill 合并成 1 个（官方形态、上下文成本、shadowing 三重反对）
- ❌ 不让编排 skill 拥有"判定门禁"的措辞——那是把 spec-kit slash 路径的坑照抄一遍
- ❌ 不新建第二份状态真相
- ❌ 不用 `AskUserQuestion` 当门禁（模型自行决定是否调用，且可超时自动放行）

待验证风险

- ☐ **project 作用域 skill frontmatter hooks 是否受 workspace trust 影响**：官方对 subagent 有明文（未 trust 则静默跳过），对 skill 只说了 `allowed-tools`。验证方式：最小复现 skill 带 `PreToolUse` hook，在未 trust 目录跑，看是否触发、是否只写 debug log
- ☐ **skill frontmatter hooks 的实际触发率**：文档说支持，但有 #31250 这类"完全不 fire"的报告。验证方式：探针断言而非目测；用 `/hooks` 确认是否登记
- ☐ **加第 8 个 skill 是否稀释路由**：本仓已有 7 个 `e2e-*` skill。验证方式：按 skill-creator 的 eval harness，同轮开"带编排 skill / 不带"两个 subagent 对照
- ☐ **listing 预算是否已接近上限**：验证方式：`/doctor` 看 listing 成本估算与最大贡献者，`/context` 看 Skills 行（预算生效后的真实值），加 skill 前后各测一次
- ☐ **本仓的服务端门禁前提是否成立**：CI/分支保护成为最终门禁的前提是执行 agent 无 admin/owner 凭据、ruleset 无 bypass actor。验证方式：`gh api repos/<owner>/<repo>/rulesets` 查 bypass 配置；核对 CI token 权限范围；行为证明（用受限身份实推一次）
- ☐ **上游漂移**：本报告 9 处 `blob/main` 链接指向可变分支，内容会漂；官方文档为 living doc 无版本号。验证方式：关键结论落地前重取一次，或改用 commit permalink 固定

- ☐ 依赖项目的存续风险：spec-kit / BMAD / agent-os / tdd-guard 的 maintainer 变动与 license 变更未评估（本次调研未覆盖）。若要直接借鉴其代码而非思路，需先查 license 与近 6 个月提交活跃度
- ☐ OCG 「审批包裹」模式：X2 报的欧洲企业实践，主 URL 已失效（curl 000），仅剩一份 vendor PDF 单源，未进入推荐依据。若要采用需重新找一手来源

主要来源

官方一手（Claude Code / Anthropic）

来源	支撑什么	置信度
https://code.claude.com/docs/en/skills	skill 生命周期、注入即常驻不重读、compaction 5k/25k 预算、listing 1% 预算与 1,536 字符截断、超预算写 debug log warning、 <code>allowed-tools</code> 受 trust 控制	高
https://code.claude.com/docs/en/hooks	30+ hook 事件、exit 2 阻断表、exit 1 非阻断陷阱、"Hooks in skills and agents" 全节、 <code>once</code> 字段	高
https://code.claude.com/docs/en/sub-agents	subagent frontmatter hooks + 未 trust 时静默跳过（限 subagent）	高
https://code.claude.com/docs/en/workflows	"who decides what runs next: The script"、" For sign-off between stages, run each stage as its own workflow "、workflow subagent 恒 acceptEdits	高
https://code.claude.com/docs/en/permissions	deny→ask→allow 求值序、hook 决策不绕过权限规则、managed settings 不可覆盖	高
https://code.claude.com/docs/en/tools-reference	AskUserQuestion 由模型自行发起、可配超时自动放行；权限提示永不自动关闭	高
https://code.claude.com/docs/en/plugins-reference	plugin 组件结构、 <code>hooks/hooks.json</code> 随插件启用生效	高
https://claude.com/blog/lessons-from-building-claude-code-how-we-use-skills	"reference other skills by name"、依赖管理未原生支持、"try to do too much... confuse the agent"	高
https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents	"One-sided evals create one-sided optimization"、"grade what the agent produced, not the path it took"	高

来源	支撑什么	置信度
https://github.com/anthropic/skills/blob/main/skills/skill-creator/SKILL.md	可直接抄的 eval harness：同轮 with/without-skill 双 subagent 对照	高

开源实现一手

来源	支撑什么	置信度
https://github.com/github/spec-kit/blob/main/workflows/speckit/workflow.yml	内置 SDD workflow 的 2 个 <code>gate</code> + <code>on_reject: abort</code>	高
https://github.com/github/spec-kit/blob/main/docs/reference/workflows.md	11 种 step 类型、gate 语义、 <code>state.json</code> 落盘、 <code>workflow resume</code>	高
https://github.com/github/spec-kit/blob/main/workflows/ARCHITECTURE.md	RunState 枚举、gate 暂停/恢复机制	高
https://github.com/github/spec-kit/blob/main/templates/commands/implement.md	slash 路径的"门禁"实为措辞："proceed with implementation anyway? (yes/no)"	中高
https://github.com/github/spec-kit/blob/main/templates/commands/analyze.md	"Recommend resolving before implement"、"Do NOT apply them automatically"	中高
https://github.com/github/spec-kit/blob/main/scripts/bash/check-prerequisites.sh	非零退出只终止自身子进程，不阻断 agent	中高
https://github.com/buildermethods/agent-os/discussions/310	Agent OS v3 退役编排与实现阶段 （反向先例）	高
https://github.com/bmad-code-org/BMAD-METHOD/releases/tag/v6.10.0	外部 orchestrator 轮询制品状态，skill 只跑单次迭代	高
https://github.com/nizos/tdd-guard/blob/main/docs/enforcement.md	真实 guardrail 的两条绕过面（改 guard 自身状态 / 走 shell 绕开被 hook 的工具）	中高

缺陷与失败报告（一手 issue）

来源	支撑什么	置信度
https://github.com/anthropics/claude-code/issues/64606	skill 描述预算 ~8,000 字符，20–25 个 skill 即触发	中高
https://github.com/anthropics/claude-code/issues/31250	blocking PreToolUse hook 完全不触发且无报错	中
https://github.com/anthropics/claude-code/issues/21988	非 2 的退出码被忽略、操作照常执行	中
https://github.com/anthropics/claude-code/issues/37210	<code>permissionDecision: deny</code> 对 Edit 工具被忽略	中
https://github.com/anthropics/claude-code/issues/44334	模型伪造 "user approved" 文字通过格式门禁	中
https://github.com/anthropics/claude-code/issues/17351	父 skill 调子 skill 后父流程丢失	中
https://github.com/github/spec-kit/issues/2459	<code>/speckit.implement</code> 不加载 constitution.md，治理断链	高

学术（标题已逐条经 arXiv 核对；前四篇为 Gemini lens 独有发现）

来源	标题与结论	置信度
https://arxiv.org/abs/2604.02375	<i>KAIJU: An Executive Kernel for Intent-Gated Execution of LLM Agents</i>	中高
https://arxiv.org/abs/2607.07405	<i>Reason Less, Verify More: Deterministic Gates Recover a Silent Policy-Violation Failure Mode...</i>	中高
https://arxiv.org/abs/2601.04688	<i>ToolGate: Contract-Grounded and Verified Tool Execution for LLMs</i>	中高
https://arxiv.org/abs/2604.22136	<i>Sovereign Agentic Loops: Decoupling AI Reasoning from Execution in Real-World Systems</i>	中
https://arxiv.org/abs/2605.01771	<i>The Compliance Gap: Why AI Systems Promise to Follow Process Instructions but Don't</i>	高
https://arxiv.org/abs/2605.24050	<i>More Skills, Worse Agents? Skill Shadowing Degrades Performance... — 52→202 skill 掉约 21%</i>	高

来源	标题与结论	置信度
https://arxiv.org/abs/2605.21384	SpecBench: Measuring Reward Hacking in Long-Horizon Coding Agents	高
https://arxiv.org/abs/2606.26924	A Deterministic Control Plane for LLM Coding Agents	中

实测与反证

来源	支撑什么	置信度
https://blog.scottlogic.com/2025/11/26/putting-spec-kit-through-its-paces-radical-idea-or-reinvented-waterfall.html	spec-kit 实测 4x 耗时、2,577 行 markdown、没有任何 checkpoint 被强制	高
https://news.ycombinator.com/item?id=45935763	"SDD: The Waterfall Strikes Back" (225 分)：做成的人靠小 spec + 验收测试	高
https://www.thoughtworks.com/en-us/insights/blog/agile-engineering-practices/spec-driven-development-unpacking-2025-new-engineering-practices	代码仍是 source of truth，仍需确定性 CI/CD；SDD 仅列 Assess	高
https://www.trychroma.com/research/context-rot	18 个模型实测：200K 窗口在 50K 就明显掉点	高
https://aipatternbook.com/approval-fatigue	门禁越多越无效；建议每 session ~10 个门禁	中高
https://docs.temporal.io/guides/reliable-document-approvers	durable execution：人审 = 流程挂起 + 外部 Signal 唤醒	高

► ► 调研 Metadata（来源处置台账 / Reflection / Citation Health / 终审辩论历史）